Pramo Jewels

Microservices / Module Architecture (If Applicable)

Document ID: PJ-MSA-001
Version: 1.0
Status: Draft

Field	Value
Project	Pramo Jewels Premium E-Commerce Platform
Purpose	Define the modular architecture today and the path to future microservices
Audience	Solution Architects, Backend Developers, DevOps Engineers, AI Coding Agents
Version	1.0

1. Architectural Recommendation

Phase 1 production should use a modular monolith built with Laravel. This provides lower operational complexity while keeping clear module boundaries. Individual modules can later be extracted into microservices if business scale justifies it.

2. Core Business Modules

Identity & Authentication
Customer Accounts
Product Catalogue
Inventory
Search
Cart & Wishlist
Checkout
Payments
Orders
Shipping & Courier
Notifications
Reviews
Coupons & Promotions
Analytics
Administration

3. Module Responsibilities

Each module owns its business logic, validation, services, repositories, events and API endpoints. Cross-module communication should occur through service interfaces and domain events rather than direct database coupling.

4. Internal Communication

Current:
• Service layer method calls
• Laravel Events & Listeners
• Queue jobs

Future:
• REST/gRPC APIs
• Event broker (RabbitMQ/Kafka) if required

5. Shared Infrastructure

MySQL, Redis, object storage, centralized logging, monitoring, configuration management and authentication are shared across modules while maintaining logical ownership.

6. Candidate Future Microservices

Authentication Service
Catalogue Service
Order Service
Payment Service
Notification Service
Shipping Service
Recommendation Service
Search Service

7. Data Ownership

Each future service should own its database schema. Cross-service data access should use APIs or events instead of direct database queries.

8. Resilience

Use retries, idempotency for payments and order creation, circuit breakers (when distributed), timeouts and graceful degradation for third-party dependencies.

9. Scalability Strategy

Scale modules independently where possible. Payment, search and catalogue services are expected to have different scaling characteristics from administrative features.

10. Migration Roadmap

Stage 1: Modular monolith.
Stage 2: Extract notification and search.
Stage 3: Extract payments and shipping.
Stage 4: Extract orders and catalogue if traffic demands.

11. Governance

Maintain API contracts, semantic versioning, centralized observability and documentation for every module or future microservice.